

گزارش تمرین کامپیوتری چهارم

سیگنال ها و سیستم ها

دکتر اخوان

ناژین نیکخواه بهرامی

۸۱۰۱۰۲۵۳۰

بهار ۱۴۰۴

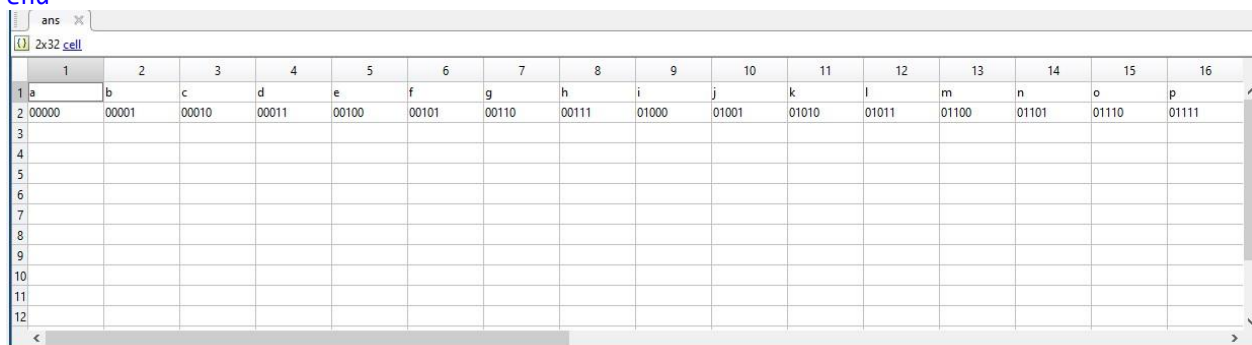
تمرین ۱-۱)

یک تابع تشکیل داده ایم که در سطر اول حروف و سطر پایین آن متناظر باینری آن را قرار میدهیم.

```
function output = create_mapset()

Nch = 32;
output = cell(2,Nch);
Alphabet = 'abcdefghijklmnopqrstuvwxyz .,;!";';
for i = 1:Nch
    output{1,i} = Alphabet(i);
    output{2,i} = dec2bin(i-1,5);
end

end
```



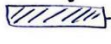
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p
2	00000	00001	00010	00011	00100	00101	00110	00111	01000	01001	01010	01011	01100	01101	01110	01111
3																
4																
5																
6																
7																
8																
9																
10																
11																
12																

تمرین ۲-۱)

باتوجه به مپ ست ساخته شده تمام کاراکتر های پیام را جداگانه ابتدا باینری میکنیم و در آرایه ذخیره میکنیم. حالا یک آرایه از بیت های پیام داریم و باید با یک روشی جداسازی شوند.

به کمک اندازه بیت ریت پیام را جدا میکنیم و عدد دسیمال متناظر را حساب میکنیم. پس از آن یک شریب ثابت بدست آمده است که باتوجه به فرکانس نمونه برداری موج سینوسی متناظر را میابیم.

For example : 

5 bit → Finding the bits with the mapset
array  Saving

For example →  We need to separate this to bits with the bit rate and give it the $0.5 \sin(wb)$.

if bit rate is 1 then each bit has a signal.
if bit rate is 2 then we separate 2-bit by 2-bit and give it a signal.

as we see there is a constant behind the $\sin(wb)$.

this depends on the bitrate : $Cons = \frac{decimal}{r^n - 1}$ (n= bitrate)

$$n=1 \rightarrow Cons = \frac{dec}{r-1} = dec \Rightarrow 0, 1$$

$$n=2 \rightarrow Cons = \frac{dec}{r-1} = \frac{dec}{r} \Rightarrow 0, \frac{1}{r}, \frac{r}{r}, 1$$

%%Part2 Coding function

function coded_sig = coding_amp(message, bit_rate)

```
fs = 100;
coded_sig = [];
mapset = create_mapset();
```

% 1- Turning the message into a binary number

```
Nch = 32;
message_len = strlen(message);
message_bin = cell(1, message_len);
for i=1: message_len
    ch = message(i);
    for j=1:Nch
        if ch == mapset{1,j}
            message_bin{i} = mapset{2,j};
        end
    end
end
binarymessage = cell2mat(message_bin);
binarymessage_len = length(binarymessage);
```

% 2- Creating the coded signal

```
for z=1: bit_rate: binarymessage_len
    bin_to_write = binarymessage(z: z+bit_rate-1);
    decimal = bin2dec(bin_to_write);
    cons = decimal / ((2^bit_rate)-1);
    t = (z/bit_rate):(1/fs):(z/bit_rate)+1-(1/fs);
    to_write = cons * sin(2*pi*(t-(z/bit_rate)));
    coded_sig = [coded_sig to_write];
end
```

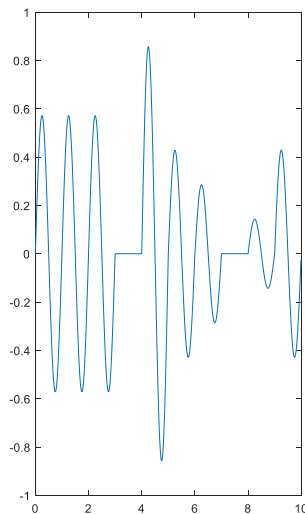
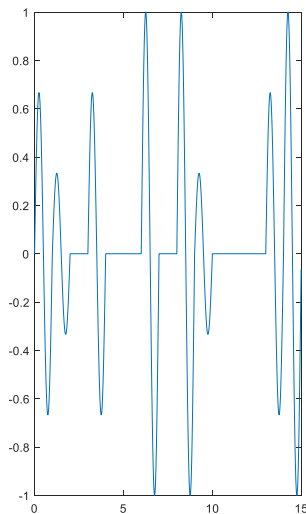
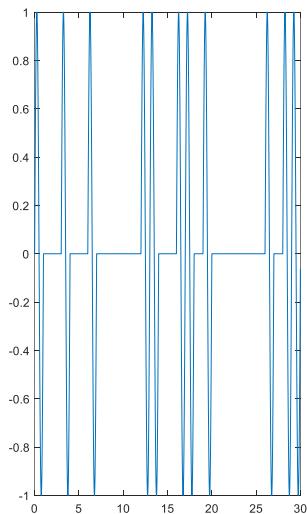
end

تمرین ۱-۳)

```
close all;
clc;
clear;
%% 1-Creating the mapset

mapset = create_mapset();
%% Part3 Coding the message

fs = 100;
message = 'signal';
coded_signal = cell(1,3);
figure
for i=1:3
    coded_signal{i} = coding_amp(message,i);
    t = 0:(1/fs):(length(coded_signal{i})/fs)-(1/fs);
    subplot(1,3,i);
    plot(t,coded_signal{i})
end
```



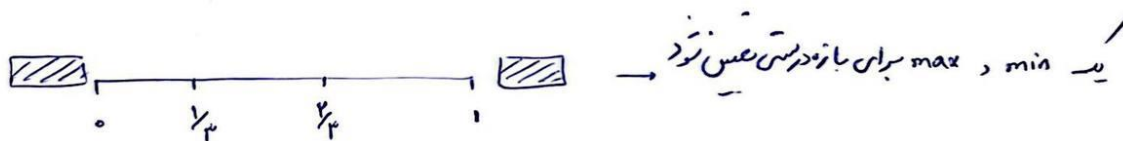
تمرین ۴-۱

در این قسمت درواقع عکس کاری که قبلا میکردیم را انجام میدهیم و ابتدا ثانیه به ثانیه سیگنال را جدا میکنیم و همپوشانی (کورلیشن) با سیگنال سینوسی میدهیم تا حاصل کورلیشن که همان ضرایب است را بدست آوریم.

حالا طبق همان توضیحات داده شده عدد دسیمال نیز بدست می آید و بعد باید عدد دسیمال را به باینری تبدیل کنیم و بعد ۵ تایی جدا کنیم. حالا یک آرایه از بیت های مختلف دسته بندی شده داریم که با استفاده از مپ کردن میتوان کاراکتر هر ۵ بیت را پیدا کرد. حالا با کنار هم قرار دادن و ذخیره سازی میتوان کاراکتر های پیام را پیدا کرد و پیام رمز گذاری شده را میابیم.

$$Constant = \frac{decimal}{r^n - 1} = \frac{dec}{r}$$

$$birate = 2 \quad \text{نسبت به ۲}$$



$$threshold = \frac{1}{r} \left(\frac{1}{r^n - 1} \right); \rightarrow \text{برای تعیین بازه شده 1/6, 2/6, 3/6}$$

```

%% Part4 Decoding function
function decoded_message = decoding_amp(coded_sig,bit_rate)

    fs = 100;
    decoded_message = [];
    mapset = create_mapset();

    % Calculating correlations and extracting binary message
    bin_message = [];
    thresholds=[];
    for k=1:(2^bit_rate)-1
        thresholds(k) = ((2*k)-1)/(2*((2^bit_rate)-1));
    end
    t = 0:(1/fs):1-(1/fs);
    ref = 2*sin(2*pi*t);
    for j=0:(length(coded_sig)/fs)-1
        to_check = coded_sig((j*fs)+1:((j+1)*fs));
        corr = 0.01*sum(to_check.*ref);
        initial_corr = thresholds(1);
        for t=1:length(thresholds)
            if corr<=thresholds(t)
                corr = ((initial_corr*2*((2^bit_rate)-1))-t)/((2^bit_rate)-1);
                break;
            elseif t==length(thresholds)
                corr = 1;
            else
                initial_corr = thresholds(t+1);
            end
        end
        bin_message = [bin_message dec2bin(round(corr*((2^bit_rate)-1)),bit_rate)];
    end

    % Turning the binary message into string
    for p = 1:5:length(bin_message)-4
        message_to_check = bin_message(p:p+4);
        message_to_check = mapset{1,bin2dec(message_to_check)+1};
        decoded_message = [decoded_message message_to_check];
    end

end

```

```

%% Part4 Decoding the signal
decoded_signal = cell(1,3);
for i=1:3
    decoded_signal{i} = decoding_amp(coded_signal{i},i);
    fprintf("bitrate = %d message = %s \n", I, decoded_signal{i});
end

```

Command Window

```

bitrate = 1 message = signal
bitrate = 2 message = signal
bitrate = 3 message = signal

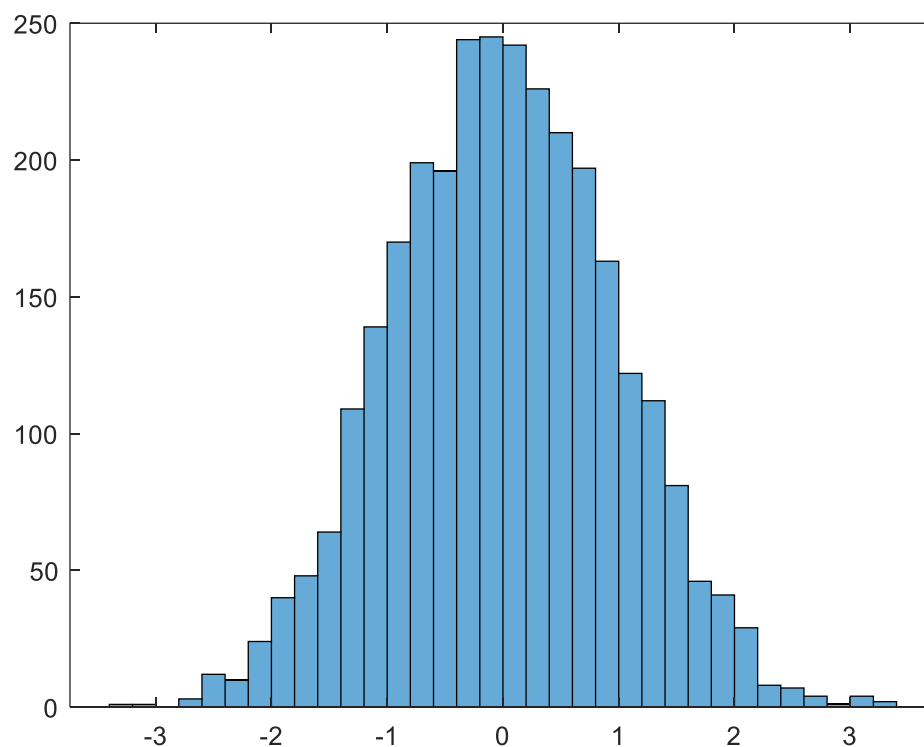
```

fx >>

تمرین ۵-۱

با استفاده از توزیع گوسی می‌خواهیم که به نمودار نویز اضافه کنیم. با استفاده از دستور زیر این کار را انجام می‌دهیم. همانطور که مشخص است میانگین ۰ و واریانس ۱ است.

```
% Part5 Adding noise  
% 5- Noise  
noise=randn(1,3000);  
figure  
histogram(noise)  
Mean=mean(noise)  
variance=var(noise)
```



Command Window

Mean =

0.0051

variance =

0.9254

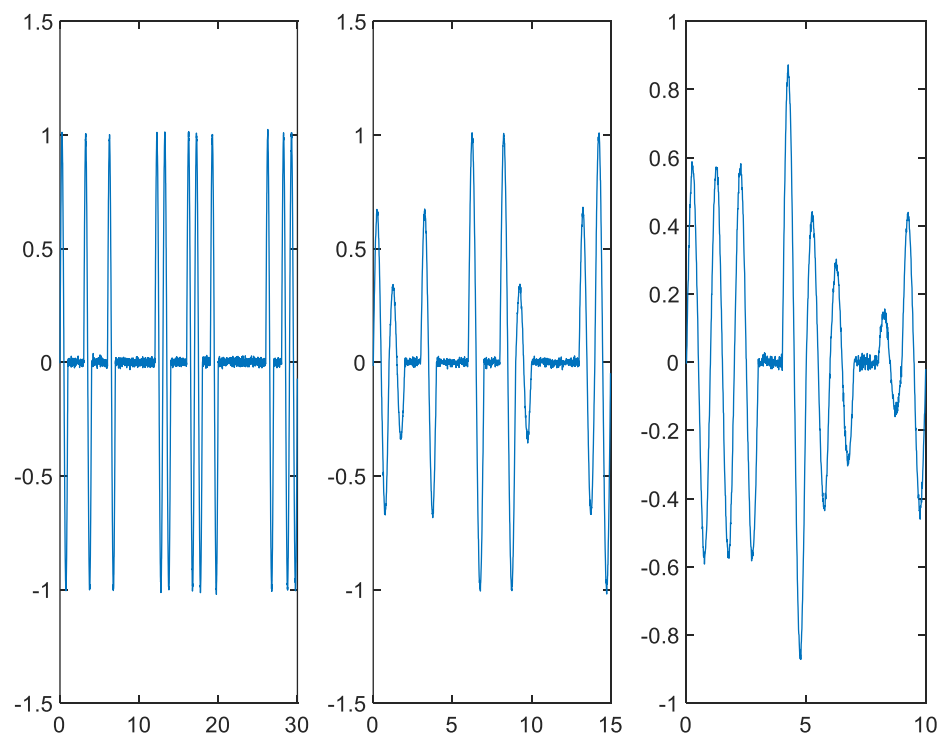
 >>

تمرین ۶-۱

چون واریانس اختلاف بین نویزها است، وقتی که این اختلاف زیاد میشود دیگر سیگنال قابل خواندن نیست و نمیتوان پیام پنهان شده را استخراج کرد.

```
% Part6 Controlling the noise
m = 0.01;

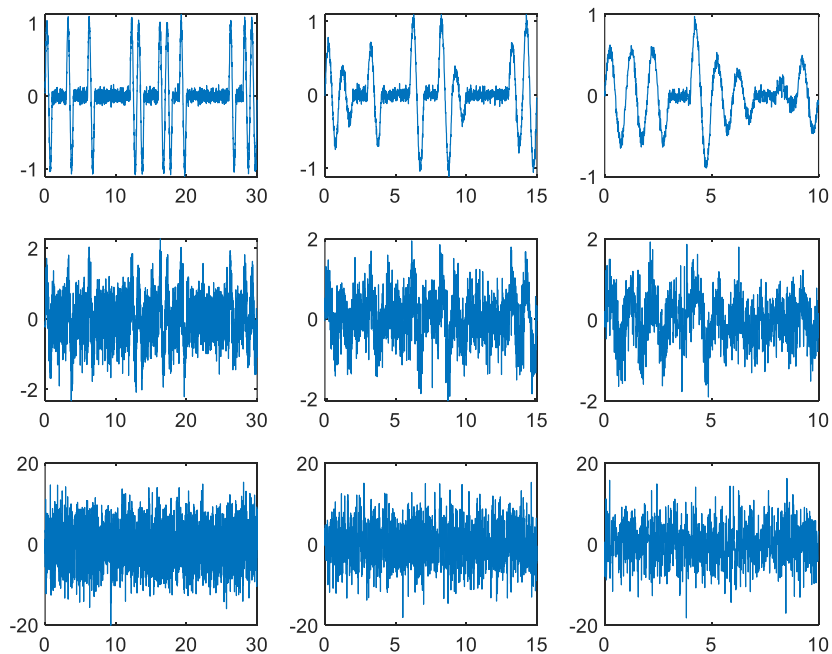
noisy_signal = cell(1,3);
for i=1:3
    noise = m * randn(1, length(coded_signal{i}));
    noisy_signal{i} = coded_signal{i} + noise;
    t = 0:(1/fs):(length(noisy_signal{i})/fs)-(1/fs);
    subplot(1,3,i);
    plot(t,noisy_signal{i})
    decoded = decoding_amp(noisy_signal{i},i);
    fprintf("bitrate = %d message = %s \n", i, decoded);
end
```



تمرین ۱-۷)

هرچه سرعت بالاتر است حساسیت به نویز بیشتر است و پیام نادرست استخراج میشود. همچنین با افزایش انحراف معیار اثر نویز بر روی سیگنال بیشتر میشود. (یه ازای ۳ انحراف معیار متفاوت، اثر نویز را بررسی کردیم)

```
% Part7 Trying different noises
M = [0.05, 0.5, 5];
figure
for j = 1:3
    m = M(j);
    fprintf('----- standard deviation = %d -----\n', m);
    for i=1:3
        noise = m * randn(1, length(coded_signal{i}));
        noisy_signal{i} = coded_signal{i} + noise;
        t = 0:(1/fs):(length(noisy_signal{i})/fs)-(1/fs);
        subplot(3,3,3*(j-1) + i);
        plot(t,noisy_signal{i})
        decoded = decoding_amp(noisy_signal{i},i);
        fprintf("bitrate = %d message = %s \n", i, decoded);
    end
end
end
```



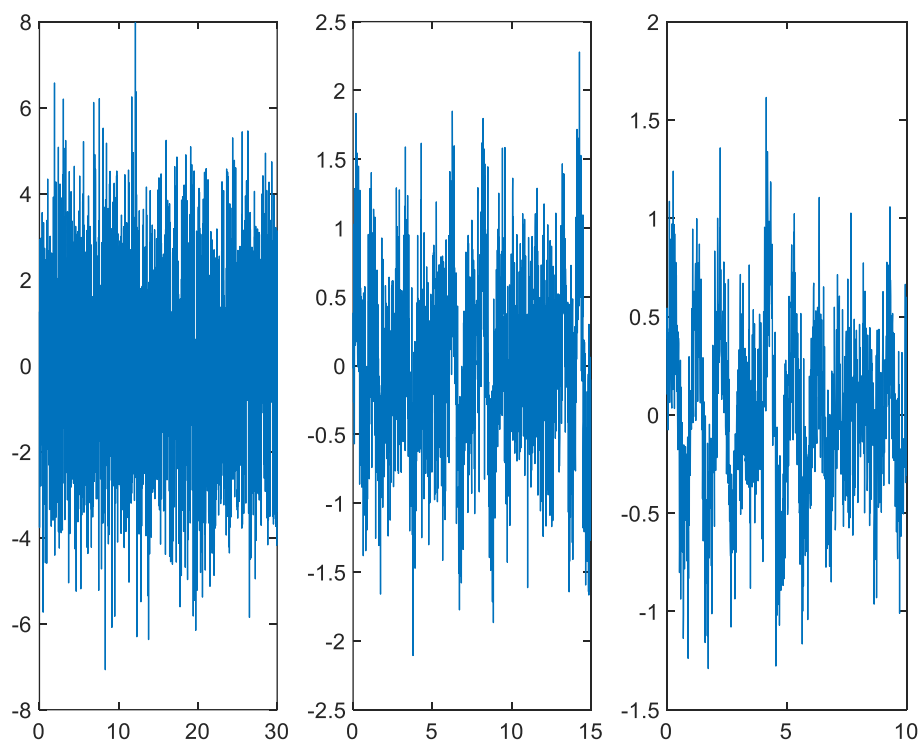
Command Window

```
----- standard deviation = 5.000000e-02 -----  
bitrate = 1 message = signal  
bitrate = 2 message = signal  
bitrate = 3 message = signal  
----- standard deviation = 5.000000e-01 -----  
bitrate = 1 message = signal  
bitrate = 2 message = signal  
bitrate = 3 message = wional  
----- standard deviation = 5 -----  
bitrate = 1 message = xioned  
bitrate = 2 message = adcmeh  
bitrate = 3 message = gx!qac
```

 >>

تمرین ۸-۱

به ازای اجرا کردن کد بعلت رندوم بودن نویز در نقاط مرزی، یا پیام خودش بدست میاید یا اینکه کمی (مثلا یه کاراکتر) تفاوت دارد.



نتیجه ۴ بار خروجی گرفتن:

```
Command Window
bitrate = 1 message = sihnal
bitrate = 2 message = signal
bitrate = 3 message = signal
bitrate = 1 message = signal
bitrate = 2 message = signal
bitrate = 3 message = sggnal
bitrate = 1 message = signaj
bitrate = 2 message = signal
bitrate = 3 message = signak
bitrate = 1 message = signal
bitrate = 2 message = signal
bitrate = 3 message = skgnal
fx >>
```

تمرین ۱-۹)

در واقع، زمانی که ما دامنه سیگنال ارسالی را افزایش می‌دهیم، به طور طبیعی آستانه‌های تشخیص ما نیز افزایش می‌یابد. این به معنی است که سیستم ما توانایی بهتری در تفکیک سیگنال اصلی از نویز خواهد داشت. به عبارت دیگر، با افزایش توان مصرفی و ارسال سیگنال با دامنه بالاتر، سیگنال ما در برابر نویز مقاومت بیشتری خواهد داشت و دقت تصمیم‌گیری‌ها بهبود می‌یابد. این به این دلیل است که یک سیگنال با دامنه بالاتر می‌تواند از پس نویزهای محیطی و تداخلات بهتر برآید، و تشخیص‌ها و شناسایی‌ها را دقیق‌تر کند.

اضافه بر این، افزایش دامنه سیگنال به معنی بهبود نسبت سیگنال به نویز است، که یکی از معیارهای به طور SNR کلیدی در سیستم‌های مخابراتی برای تعیین کیفیت و قابلیت اطمینان ارتباط است. بهبود مستقیم منجر به کاهش خطاهای احتمالی در انتقال داده‌ها و افزایش قابلیت اطمینان ارتباط می‌شود. بنابراین، در بسیاری از موارد افزایش دامنه و توان سیگنال ارسالی به عنوان یک راهکار موثر برای بهبود عملکرد سیستم‌های مخابراتی و مقاوم‌سازی آن‌ها در برابر نویز و تداخلات محیطی در نظر گرفته می‌شود.

تمرین ۱-۱۰)

در صورت عدم وجود نویز در سیستم کدینگ دامنه، بیت‌ریت می‌تواند به طور تئوری به بی‌نهایت نزدیک شود. به عبارت دیگر، در یک محیط بدون نویز، محدودیتی برای افزایش بیت‌ریت وجود ندارد. البته، در عمل موارد دیگری مانند محدودیت‌های سخت‌افزاری، پهنای باند کانال و محدودیت‌های فیزیکی سیستم‌ها تأثیرگذار خواهند بود.

۱. پهنای باند کانال: حتی در صورت عدم وجود نویز، پهنای باند کانال محدودیتی برای افزایش بیت‌ریت ایجاد می‌کند. ظرفیت کانال بر اساس پهنای باند و تکنیک‌های مدولاسیون استفاده شده تعیین می‌شود.
۲. محدودیت‌های سخت‌افزاری: سرعت پردازش و ذخیره‌سازی داده‌ها نیز نقشی مهم در تعیین بیت‌ریت دارد. حتی اگر نویزی نباشد، سخت‌افزار باید بتواند با سرعت بالای بیت‌ریت هماهنگ شود.

تمرین ۱-۱۱)

در اصل، اهمیت دامنه سیگنال ارسالی به مراتب بیشتر از آن است که سیگنال دریافتی با چه سیگنالی برای عملیات correlation یا ضرب شود. سیگنال ارسالی باید قدرت کافی داشته باشد تا در مقابل نویز محیطی مقاومت کند.

به این ترتیب، اگر سیگنال ارسالی دارای دامنه کافی نباشد، حتی اگر سیگنال دریافتی را با اعداد مختلف ضرب کنیم، در نهایت تأثیر نویز کاهش نخواهد یافت. به عبارتی دیگر، زمانی که سیگنال از فرستنده خارج می‌شود، اگر نویز محیطی از حدی بیشتر باشد، سیگنال اصلی دیگر قابل تشخیص نخواهد بود. اعداد ۲ و ۱۰ که برای ضرب استفاده شده‌اند، تنها برای راحتی در انجام عملیات ریاضی و بدون تغییر در اصول پایه‌ای تأثیر نویز بر سیگنال است.

تمرین ۱-۱۲)

سرعت اینترنت‌های ADSL معمولاً بین ۸ تا ۱۶ مگابیت بر ثانیه است، که معادل حدود ۸ میلیون بیت در ثانیه می‌باشد. در این تمرین، ما با سرعت‌هایی مانند ۱ بیت در ثانیه، ۲ بیت در ثانیه و ۳ بیت در ثانیه داده‌ها را ارسال کردیم. این تفاوت عظیم در سرعت ارسال داده‌ها نشان می‌دهد که تمرین ما در مقایسه با سرعت‌های واقعی اینترنت بسیار آهسته‌تر است.

این مثال می‌تواند به وضوح نشان دهد که در مقیاس‌های کوچک‌تر و سرعت‌های پایین‌تر، چگونه مشکلات انتقال داده‌ها و تأثیر نویز بررسی و تحلیل می‌شود، اما در دنیای واقعی اینترنت و ارتباطات پرسرعت، مسائل بسیار پیچیده‌تر هستند.

پایان.